

BAB 1

KONSEP PEMROGRAMAN BERORIENTASI OBJEK

I. Tujuan Instruksional Khusus

Setelah Menyelesaikan praktek ini mahasiswa dapat :

1. Menyebutkan dan menjelaskan karakteristik pemrograman berorientasi objek
2. Menjelaskan perbedaan objek dan kelas
3. Membuat diagram objek dan kelas
4. Menjelaskan tentang instance variabel dan metode
5. Membuat kelas dalam Java
6. Menerapkan konsep pemrograman berorientasi objek kedalam program Java

II. Persyaratan / Kebutuhan Sistem Komputer

Persyaratan / kebutuhan sistem untuk menjalankan program Java adalah sebagai berikut :

1. Processor P-II atau yang lebih tinggi,
2. Ram minimal 32 Mbyte,
3. Free space harddisk minimal 200 Mbyte,
4. Mouse (Serial / PS2),
5. Sistem Operasi :
 - a. Minimal Windows 98 (disarankan versi Windows yang lebih tinggi),
 - b. Linux.

III. Dasar Teori

Konsep pemrograman berorientasi objek

Pemrograman Berorientasi Objek (PBO) atau *Object Oriented Programming* (OOP) adalah merupakan inti (*core*) dari bahasa Java, yang bukan bersifat opsional. Konstruksi dari pemrograman Java selalu berbasis pada OOP. OOP ini merupakan sebuah teknik pemrograman yang bertujuan untuk mengurangi kompleksitas suatu program, agar lebih sederhana dan mudah.

Karakteristik OOP

Elemen dasar dari *object-oriented programming* adalah abstraksi (*abstraction*). Secara alami manusia dalam melihat atau mengidentifikasi suatu benda berdasarkan pada pandangan abstraksi. Pandangan abstraksi ini hanya melihat pada bentuk dan fungsi umumnya. Selain abstraksi ada beberapa metode untuk menentukan karakteristik dalam pendekatan berorientasi objek, yaitu:

Identifikasi

Yaitu bahwa data yang diukur mempunyai nilai/sifat tertentu secara individual, yang membedakan entitas satu dengan entitas lainnya disebut objek. Objek dalam OOP sering disebut sebagai *instance* dari sebuah kelas yang merupakan anggota dari kelas yang mempunyai sifat yang khusus sesuai dengan kelasnya. Sebagai contoh identifikasi terhadap objek binatang: sapi, kerbau, ular dan buaya.

Klasifikasi

Adalah suatu kegiatan mengumpulkan data (*atribut/variabel*) dan metode/perilaku (operasi) dari suatu objek yang mempunyai struktur data yang sama kedalam satu grup yang disebut kelas (*class*).

Objek dan Kelas

Pada pemrograman berorientasi objek, yang disebut objek (*objects*) adalah instant dari sebuah kelas (*class*), kelas terdiri dari variabel/atribut dan fungsi/metode yang dibungkus dengan mekanisme pembungkusan kelas.

Enkapsulasi (*encapsulation*)

Enkapsulasi adalah merupakan dasar untuk pembatasan ruang lingkup program terhadap data yang diproses. Enkapsulasi ini mencegah dan melindungi kode dan data dari akses dari kode lain, yaitu dengan cara menyembunyian informasi (*information/data hiding*), dimana variabel/data tidak dapat diakses oleh metode/fungsi diluar kelas. Mekanisme utamanya setiap variabel yang digunakan didalam kelas biasanya bersifat private.

Penurunan Kelas (*inheritance*)

Inheritance adalah proses penciptaan kelas baru dari kelas sebelumnya, yang disebut kelas turunan (*derived classes/child class*) atau subkelas. Kelas turunan akan mewarisi semua data/atribut dan metode dari kelas induknya (*base class/parent class/superclass*).

Polymorphism

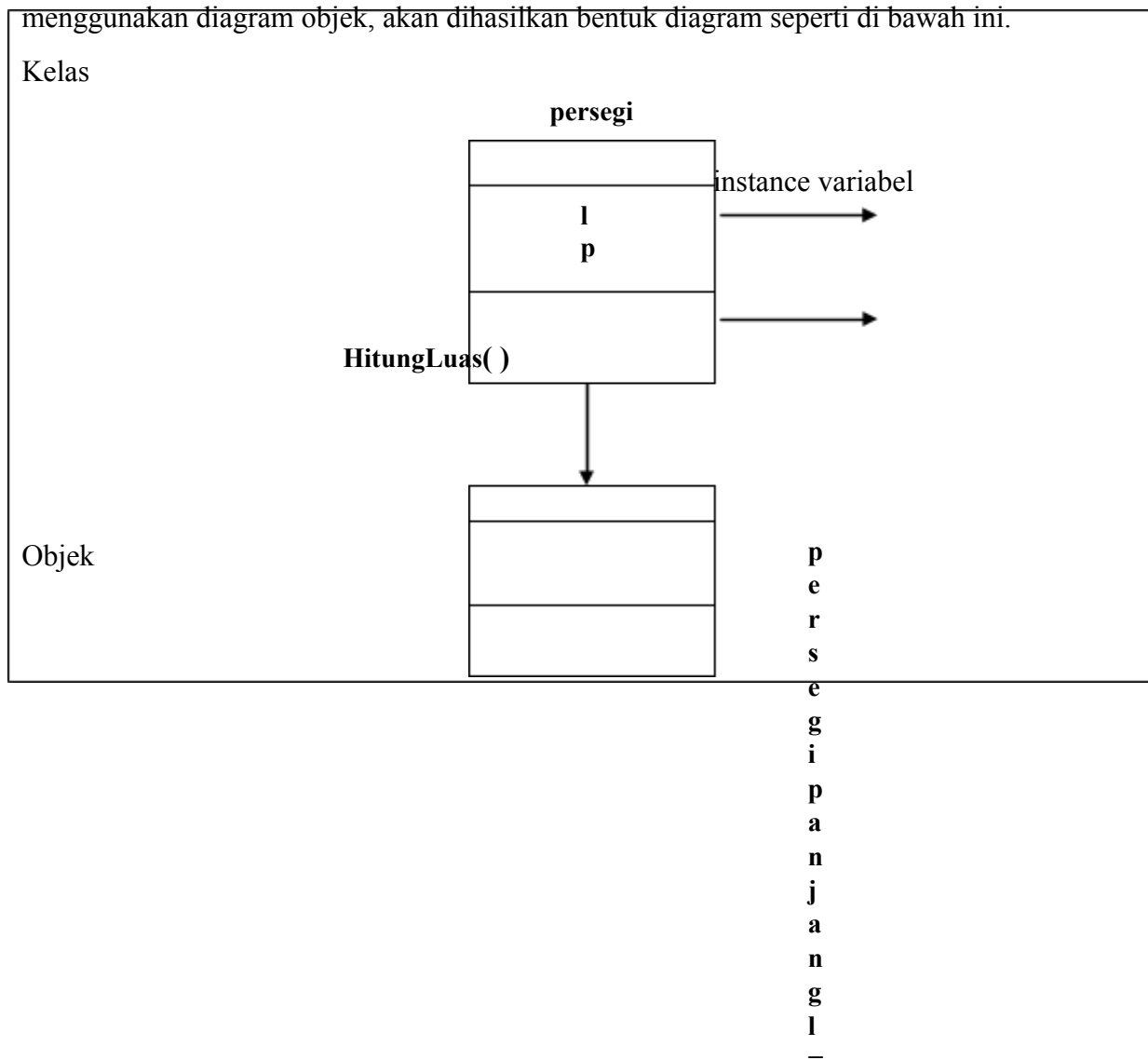
Polymorphism (berasal dari bahasa Yunani yang artinya adalah “macam-macam bentuk” “*many forms*”) adalah sebuah konsep yang menyatakan bahwa sesuatu yang sama dapat mempunyai bentuk dan perilaku yang berbeda.

Metode *overloading* adalah salah satu cara penerapan dalam konsep **polimorfisme**.

Kelas (*class*) dan Objek (*object*)

Pada pemrograman berorientasi objek, yang disebut objek adalah instant dari sebuah kelas. Kelas terdiri dari variabel/data dan metode yang dibungkus dengan mekanisme pembungkusan kelas. Ketika mendefinisikan kelas, berarti juga mendeklarasikan dan menentukan data dan metode yang mengoperasikan/manipulasi data tersebut. Sebuah kelas yang sederhana hanya berisi sebuah data dan metode.

Sebagai contoh kita akan menghitung luas dari objek persegi panjang, maka dengan menggunakan diagram objek, akan dihasilkan bentuk diagram seperti di bawah ini.



Gambar 2.1 Diagram objek dan kelas persegi

Mendeklarasikan kelas

Sebuah kelas dideklarasikan dengan menggunakan kata kunci ***class***. kelas ini merupakan objek template yang berisi spesifikasi umum dari suatu objek.

Bentuk umum dari sintak class adalah sebagai berikut:

class namakelas

```

{
    tipe variable; //instance-variable1;
    namametode-1(parameter)// instance metode
    {
        //kode program
    }

    namametode-2(parameter)// instance metode
    {
        // kode program
    }
}

```

Data atau variabel yang dideklarasikan dalam kelas disebut sebagai *instance variables*. Metode (*methode*) merupakan bagian dari kode program yang dibuat untuk melaksanakan tugas/fungsi tertentu.

Mendeklarasikan sebuah objek

Pada saat kita membuat sebuah kelas (*objek template*), maka kita juga membuat tipe data baru. Tipe ini bisa digunakan untuk mendeklarasikan objek sesungguhnya. Untuk menggunakan tipe variable ini kita harus:

1. Mendeklarasikan tipe variabel kelas
2. Membuat copy terhadap variable tersebut secara fisik, dengan menggunakan operator ***new***. Operator ini secara dinamis mengalokasikan memori untuk objek dan membuat referensi ke objek Man.

Perhatikan statemen berikut ini untuk mendeklarasikan objek.

```
persegi persegipanjang1 = new persegi ( );
```

Mendeklarasikan metode

Pada intinya sebuah kelas terdiri dari *instance variabel* dan metode (*methode*). Metode adalah merupakan bagian dari kode program yang dibuat untuk melaksanakan tugas/fungsi tertentu. Suatu metode yang tidak mengembalikan nilai harus bertipe ***void***. Sedang metode yang

mengembalikan nilai digunakan statemen *return value*, dimana value adalah nilai yang dikembalikan.

Tipe Acces Modifier

Tipe public

Kata kunci `public` adalah sebuah *access specifier/modifier*, dimana mengijinkan programmer untuk mengontrol `visibility` dari variabel, metode dan class.. Ketika sebuah class member dimulai dengan `public`, maka member itu dapat diakses oleh kode diluar class dimana itu dideklarasikan.

Tipe private

Kebalikan dari `public` adalah `private`, dimana mencegah dari penggunaan oleh kode diluar kelasnya. Variable atau metode dengan tipe `private` tidak dapat diakses oleh objek lainnya.

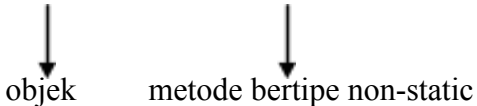
Tipe static

Manfaat dari penggunaan `static` adalah bahwa metode atau variable tersebut dapat diakses tanpa harus membuat sebuah instance dari sebuah kelas (objek). Metode `static` dapat diakses dengan menyebutkan nama kelas diikuti titik (.) lalu nama metode.

Tipe non-static

Tipe non-static hanya dapat dipanggil pada sebuah objek. Metode ini bersifat default, artinya jika sebuah metode tidak didefinisikan `static`, maka otomatis akan bersifat non-static. Sebagai contoh lihat potongan kode program berikut ini.

```
persegi panjang.HitungLuas( );
```



objek metode bertipe non-static

Metode `HitungLuas( )` tersebut dipanggil pada sebuah objek `persegi panjang`, tidak bisa dipanggil tanpa melalui sebuah objek, seperti berikut ini.

```
HitungLuas( );
```

IV. Langkah Kerja

Jalankan Edit Plus, buat file baru java dan ketikkan kode program berikut :

1. Buat program menggunakan metode dan parameter.

//DENGAN MENGGUNAKAN METODE DAN PARAMETER

```
class Man
```

```
{ //class hanya membuat objek template
    //instance variabel
    String nama;
    double gapok;
    int masakerja;
    //metode pengembalian nilai
    int hitung_gaji()
    {
        return gapok*masakerja;
    }
    // Sets dimension Man
    void setDim(String nm, double g, int mk )
    {
        Nama=nm;
        gapok=g;
        masakerja=Mk;
    }
}
```

```
class KaryawanMetodeParameter
```

```
{
    public static void main(String[] args)
    {
        Man KaryawanKu = new Man();//membuat objek sesungguhnya
        Man KaryawanMu = new Man(); //membuat objek sesungguhnya
        double gaji;
        //nilai instance variabel-1
        // penetapan nilai awal
        KaryawanKu.setDim("Eko,1000,10);
        gaji=KaryawanKu.hitung_gaji();
        System.out.println("Karyawan Ku:"+KaryawanKu.nama);
        System.out.println("Gajinya:"+gaji);
        //nilai instance variabel-2
```

```

        KaryawanMu.setDim("Dwi",2000,10);
        gaji=KaryawanMu.hitung_gaji();
        System.out.println("Karyawan Mu:"+KaryawanMu.nama);
        System.out.println("Gajinya:"+gaji);
    }
}

```

2. Sebagai contoh di bawah ini diberikan sebuah program menggunakan konsruktor tanpa parameter.

```
class Man
```

```

{
    String nama;
    double gapok;
    int masakerja;
    // konstruktor tanpa parameter
    Man()
    {
        System.out.println("Konstruksi dari Man");
        nama="Eko";
        gapok=1000;
        masakerja=10.5;
    }

    double hitung_gaji();
    {
        return gapok*masakerja;
    }
}

```

```
class KonstruktorKu
```

```

{
    public static void main(String args[])
    {

```

Man KaryawanKu = new Man()

Man KaryawanMu = new Man()

```

        double gaji;
        gaji = KaryawanKu.hitung_gaji();
        System.out.println("Volume is " + gaji);
        gaji = KaryawanMu.hitung_gaji();
        System.out.println("Volume is " + gaji);
    }
}

```

3. Penerapan parameter ke konstruktor

class Man

```

{
    String nama;
    double gapok;
    int masakerja;
    Man(String nm, double g, int mk) // konstruktor Man.
    {
        nama=nm;
        gapok=g;
        masakerja=mk;
    }

    // metode untuk menghitung gaji dan me-return
    double hitung_gaji()
    {
        return gapok*masakerja;
    }
}

```

class KonstruktorParameter

```

{
    public static void main(String args[])

```

```
{
```

```
// deklarasi, alokasi, dan inisialisasi objek
```

```

Man KaryawanKu = new Man("eko",10000,10);
Man KaryawanMu = new Man("Dwi",20000,10);
double gaji;

//ambil gaji dari KaryawanKu
gaji = KaryawanKu.hitung_gaji();
System.out.println("Gaji Karyawanku " + gaji);

// ambil gaji dari KaryawanMu
gaji = KaryawanMu.hitung_gaji();
System.out.println("Gaji Karyawanmu" + gaji);
}
}

```

4. Program Simpan program Java di atas.
5. Eksekusi program Java Pertama dengan mengklik *Tool* □ *Compile Program Java*. Lalu
Tool □ *Menjalankan Program Java*.
6. Buat analisa terhadap proses program tersebut sebelum dijalankan dan pada saat dijalankan terhadap parameter dan argument yang digunakan..

V. Pertanyaan

1. Jelaskan apa yang dimaksud dengan pemrograman berorientasi objek
2. Sebutkan karakteristik pemrograman berorientasi objek
3. Jelaskan apa yang dimaksud objek dan kelas dalam OOP.
4. Jelaskan apa yang dimaksud enheritance, polymorfism dan encapsulation
5. Buat sebuah diagram objek untuk menjelaskan sifat inheritance dalam OOP
6. Sebutkan dan jelaskan tipe acces modifier.
7. Sebuah sepeda motor, mempunyai kecepatan 50 Km per jam, dan konsumsi BBM yang digunakan 40 Km per liter. Jika sepeda motor tersebut digunakan 15 jam berapa jarak yang dapat ditempuh oleh kendaraan tersebut dan berapa liter BBM yang digunakan
8. Jelaskan tiga tahapan pelewatan nilai melalui parameter dalam metode
9. Buat program untuk menghitung gaji karyawan, dimana gaji total= (gaji pokok *

masakerja) + bonus.

bonus dihitung dengan ketentuan, sebagai berikut:

- pegawai Gol III bonus= $10\% \times \text{gaji pokok}$
- pegawai Gol II bonus= $15\% \times \text{gaji pokok}$
- pegawai Gol I bonus= $20\% \times \text{gaji pokok}$

BAB 2

METODE DAN KONSTRUKTOR

I. Tujuan Instruksional Khusus

Setelah Menyelesaikan praktek ini mahasiswa dapat :

1. Menjelaskan pengertian metode, konstruktor, parameter dan argument.

2. Mahasiswa dapat membuat program menggunakan metode dengan parameter dan argumen
3. Mahasiswa dapat membuat program dengan menggunakan konstruktor.

II. Persyaratan / Kebutuhan Sistem Komputer

Persyaratan sistem untuk menjalankan program Java adalah sebagai berikut :

1. Processor P-II atau yang lebih tinggi,
2. Ram minimal 32 Mbyte,
3. Free space harddisk minimal 200 Mbyte,
4. Mouse (Serial / PS2),
5. Sistem Operasi :
 - a. Minimal Windows 98 (disarankan versi Windows yang lebih tinggi),
 - b. Linux.

III. Dasar Teori

3.3.1 Membuat metode dengan parameter dan argumen

Parameter adalah sebuah variable yang didefinisikan oleh metode yang menerima sebuah nilai.

Sedangkan **Argumen** adalah sebuah nilai yang dilewatkan ke metode ketika metode dipanggil.

Bentuk umum dari sintaks **metode** dengan **parameter** adalah sebagai berikut:

Nama metode(*parameter*)

```
{
    // kode program
}
```

3.3.2 Konstruktor

Konstruktor merupakan sebuah fungsi (metode) yang digunakan untuk menginisialisasikan suatu objek baru. Fungsi ini dijalankan secara otomatis pada saat kelas diciptakan, dengan **nama yang sama dengan kelasnya, dan tidak mengembalikan nilai**, tetapi tidak menggunakan fungsi **void**. Konstruktor dipanggil ketika suatu objek dibuat berdasarkan kelasnya. Dari bentuk pendeklarasian, konstruktor dapat dibedakan menjadi dua, yaitu:

- Konstruktor implicit: didefinisikan oleh Java, dan disebut: *default constructor*
- Konstruktor eksplisit: ditulis oleh programmer.

Dari penggunaan parameter konstruktor ada dua jenis:

- Konstruktor dengan parameter
- Konstruktor tanpa parameter

Seperti kita ketahui, ketika kita mendeklarasikan sebuah objek, kita menggunakan bentuk umum sebagai berikut:

```
Namakelas var = new namakelas( );
```

Parentheses () diperlukan sesudah nama kelas, sesungguhnya menunjukkan bahwa konstruktor sedang dipanggil. Sehingga statemen berikut ini

```
persegi persegipanjang = new persegi ( );
```

new persegi() adalah sedang memanggil konstruktor **persegi()**. Jika kita tidak mendefinisikan sebuah konstruktor secara eksplisit untuk sebuah kelas, maka Java membuat sebuah *default konstruktor* untuk kelas tersebut. Hal inilah yang membuat mengapa program yang terdahulu ‘program pertama **class NonOOP**’ yang tidak mendefinisikan konstruktor secara eksplisit tetap berjalan. *Default konstruktor* secara otomatis menginisialisasi semua instance variables bernilai **noI**. Namun demikian *default konstruktor* hanya cocok untuk kelas-kelas yang sederhana.

Konstruktor tanpa parameter

Konstruktor ini digunakan jika objeknya mempunyai dimensi/ukuran yang sama.

Konstruktor menggunakan parameter

Ketika metode konstruksi **Man()** dalam contoh di atas diinisialisasi sebuah Objek, hal ini **sangat tidak bermanfaat**, karena semua **Man** mempunyai dimensi yang sama. Bagaimana caranya jika obyeknya mempunyai dimensi yang berbeda-beda? Solusinya adalah dengan menerapkan parameter ke konstruktor. Parameter ini merupakan variable lokal yang digunakan pada konstruktor dan dapat menggunakan nama yang sama dengan instance variable, tetapi biasanya lebih mudah menggunakan nama yang berbeda. Jika menggunakan nama yang sama maka variable lokal akan menyembunyikan ‘hide’ instance variable, dan hal ini dapat menyebabkan terjadinya *collision* antara instance variable dengan variable lokal. Untuk mengatasi hal ini maka dapat menggunakan kata kunci **this**.

IV. Langkah Kerja

Jalankan Edit Plus, buat file baru java dan ketikkan kode program berikut :

1. Buat program menggunakan metode dan parameter.

//DENGAN MENGGUNAKAN METODE DAN PARAMETER

```
class Man
```

```
{ //class hanya membuat objek template
```

```
    //instance variabel
```

```
    String nama;
```

```
    double gapok;
```

```
    int masakerja;
```

```
    //metode pengembalian nilai
```

```
    int hitung_gaji()
```

```
    {
```

```
        return gapok*masakerja;
```

```
    }
```

```
    // Sets dimension Man
```

```
    void setDim(String nm, double g, int mk )
```

```
    {
```

```
        Nama=nm;
```

```
        gapok=g;
```

```
        masakerja=Mk;
```

```
    }
```

```
}
```

```
class KaryawanMetodeParameter
```

```
{
```

```
    public static void main(String[] args)
```

```
    {
```

```
        Man KaryawanKu = new Man();//membuat objek sesungguhnya
```

```
        Man KaryawanMu = new Man(); //membuat objek sesungguhnya
```

```
        double gaji;
```

```
        //nilai instance variabel-1
```

```
        // penetapan nilai awal
```

```
KaryawanKu.setDim("Eko,1000,10);  
gaji=KaryawanKu.hitung_gaji();
```

```

        System.out.println("Karyawan Ku:"+KaryawanKu.nama);
        System.out.println("Gajinya:"+gaji);
        //nilai instance variabel-2
        KaryawanMu.setDim("Dwi",2000,10);
        gaji=KaryawanMu.hitung_gaji();
        System.out.println("Karyawan Mu:"+KaryawanMu.nama);
        System.out.println("Gajinya:"+gaji);
    }
}

```

2. Sebagai contoh di bawah ini diberikan sebuah program menggunakan konsruktor tanpa parameter.

```

class Man
{
    String nama;
    double gapok;
    int masakerja;
    // konstruktor tanpa parameter
    Man()
    {
        System.out.println("Konstruksi dari  Man");
        nama="Eko";
        gapok=1000;
        masakerja=10.5;
    }

    double hitung_gaji();
    {
        return gapok*masakerja;
    }
}

```

```
class KonstruktorKu
```

```
{
```

```

    public static void main(String args[])
    {
        Man KaryawanKu = new Man()
        Man KaryawanMu = new Man()
        double gaji;
        gaji = KaryawanKu.hitung_gaji();
        System.out.println("Volume is " + gaji);
        gaji = KaryawanMu.hitung_gaji();
        System.out.println("Volume is " + gaji);
    }
}

```

3. Penerapan parameter ke konstruktor

class Man

```

{
    String nama;
    double gapok;
    int masakerja;
    Man(String nm, double g, int mk) // konstruktor Man.
    {
        nama=nm;
        gapok=g;
        masakerja=mk;
    }

    // metode untuk menghitung gaji dan me-return
    double hitung_gaji()
    {
        return gapok*masakerja;
    }
}

```

class KonstruktorParameter

PAGE 2

{

public static void main(String args[])

```

{
    // deklarasi, alokasi, dan inialisasi objek
    Man KaryawanKu = new Man("eko",10000,10);
    Man KaryawanMu = new Man("Dwi",20000,10);
    double gaji;

    //ambil gaji dari KaryawanKu
    gaji = KaryawanKu.hitung_gaji();
    System.out.println("Gaji Karyawanku " + gaji);

    // ambil gaji dari KaryawanMu
    gaji = KaryawanMu.hitung_gaji();
    System.out.println("Gaji Karyawanmu" + gaji);
}
}

```

4. Program Simpan program Java di atas.
5. Eksekusi program Java Pertama dengan mengklik **Tool ▢ Compile Program Java**. Lalu

Tool ▢ Menjalankan Program Java.

6. Buat analisa terhadap proses program tersebut sebelum dijalankan dan pada saat dijalankan terhadap parameter dan argument yang digunakan..

V. Pertanyaan

1. Jelaskan pengertian metode, konstruktor, parameter dan argumen.
2. Sebutkan jenis-jenis konstruktor dan berikan contoh pemakaian dalam program
3. Buat program menggunakan konstruktor tanpa parameter untuk menghitung rata-rata BBM yang digunakan oleh kendaraan dalam waktu 1 minggu, jika per hari menghabiskan 3 lt kecuali untuk hari sabtu dan minggu menghabiskan 5 lt.
4. Untuk kasus yang sama jika jenis BBM-nya adalah:
 - Bensin
 - Solar
 - biodiesel

BAB 3

OVERLOADING & OVERRIDING

I. Tujuan Instruksional Khusus

Setelah menyelesaikan praktek ini mahasiswa dapat :

1. menjelaskan pengertian konsep *polimorfisme*, *overloading* dan *overriding*,
2. menjelaskan jenis metode *overloadin*,
3. dapat membuat program dengan mengimplementasikan konsep *polimorfisme*.

II. Persyaratan / Kebutuhan Sistem Komputer

Persyaratan sistem untuk menjalankan program Java adalah sebagai berikut :

1. processor P-II atau yang lebih tinggi,
2. ram minimal 32 mbyte,
3. free space harddisk minimal 200 mbyte,
4. mouse (serial / ps2),
5. sistem operasi :
 - a. minimal Windows 98 (disarankan versi Windows yang lebih tinggi),
 - b. linux.

III. Dasar Teori

Polimorfisme

Polymorphism (berasal dari bahasa yunani (*Greek*) yang artinya adalah macam-macam bentuk (*many forms*) adalah sebuah konsep yang menyatakan bahwa sesuatu yang sama dapat mempunyai bentuk dan perilaku yang berbeda. *overloading* adalah salah satu cara penerapan dalam konsep polimorfisme. *Overloading* ada dua jenis, yaitu:

1. *Overloading methode*
2. *Overloading constructor*

Overloading methode

Dalam Java memungkinkan untuk mendefinisikan dua atau lebih metode dalam kelas yang sama, menggunakan nama yang sama, tetapi parameter yang dideklarasikan harus berbeda (jumlah atau tipe parameternya). Jika jumlah parameternya sama maka Tipe harus berbeda semua, tetapi jika jumlah parameternya berbeda-beda maka boleh tipenya sama. Jenis-jenis metode overloading:

1. Jumlah parameter beda, tipe beda
2. Jumlah parameter sama, tipe beda
3. Jumlah parameter beda, tipe sama

Metode overloading ini, terjadi pada objek yang sama, tapi nilai argumennya berbeda-beda.

Ketika metode **overloaded** dipanggil, Java mencari kesamaan antara argumen yang digunakan untuk memanggil metode dan parameter dari metode. Tetapi, kesamaan ini tidak mutlak. Hal ini karena pada java adanya otomatisasi konversi tipe yang berbeda.

//PENERAPAN KONVERSI OTOMATIS PADA METODE OVERLOADING

```
class OverloadMetode
{
    //konversi tujuan bertipe integer
    void test(int a)
    {
        System.out.println("Hasil konversi byte ke integer : " + a);
    }
}

class OverloadKonversi
{
    public static void main(String args[])
    {
        OverloadMetode ob = new OverloadMetode();
        //variabel asal yang akan dikonversi bertipe byte
        byte i= 88;
        ob.test(i); // memanggil test(integer)
    }
}
```

Konversi otomatis pada java seperti program diatas, hanya dapat dilakukan jika kedua aturan ini dipenuhi:

1. Kedua tipe data (data asal dan tujuan) memiliki kompatibilitas(*compatible*).
2. Tipe tujuan mempunyai range lebih besar daripada tipe asal.

Selain itu Java juga mengerjakan konversi secara otomatis ketika menyimpan konstanta integer literal menjadi variable bertipe **byte**, **short**, atau **long**.

Overloading constructors

Overloading selain diterapkan pada **metode** juga diterapkan pada **konstuktur**. Bedanya *overloading constructor* terjadi pada objek yang berbeda-beda, nilai argument berbeda,

sehingga metode bisa lebih dari satu. Penerapan *overloading* pada kelas Java akan menjadi suatu norma tanpa perkecualian.

Method Overriding

Metode overriding adalah metode pada subclass yang mempunyai nama dan tipe identik/sama dengan metode superclass. Jika kita ingin memanggil metode superclass pada subclass dapat menggunakan statemen berikut:

Super.metode();

Metode() merupakan nama metode superclass yang dipanggil.

IV. Langkah Kerja

Jalankan Edit Plus, buat file baru java dan ketikkan kode program berikut :

1. Buat program OVERLOAD JUMLAH PARAMETER BEDA, TIPE BEDA

class OverLoadMetode

```
{
    //overload tes pertama
    void tes()
    {
        System.out.println("Tanpa Parameter");
    }

    //overload kedua
    void tes(int a)
    {
        System.out.println("nilai a:"+a);
    }

    //overload ketiga
    void tes(int a,int b)
    {
        System.out.println("Nilai a dan b = "+a+" dan "+b);
    }
}
```

}

```

        //overload ke empat
double tes(double c)
{
    System.out.println("Nilai c="+c);
    return c*c;
}
}

class OverLoad1
{
    public static void main(String[] args)
    {
        OverLoadMetode demo = new OverLoadMetode();
        double kali;
        //Memanggil semua versi tes
        demo.tes();      demo.tes(10);      demo.tes(5,8);
        kali=demo.tes(10.0);
        System.out.println("Hasil (c*c) =" +kali);
    }
}

```

2. Sebagai contoh di bawah ini diberikan sebuah program menggunakan OVERLOAD DENGAN JUMLAH PARAMETR SAMA, TIPE BEDA .

```

class OverLoadMetode
{
    //overload kesatu
    void tes(double a)
    {
        System.out.println("Nilai a bertipe double =" +a);
    }

    //overload ke dua
    int tes(int b)

```

{

```

        System.out.println("Nilai b bertipe integer="+b);
    return b+b;
    }
}

class OverLoad2
{
    public static void main(String[] args)
    {
        OverLoadMetode demo = new OverLoadMetode();
        int jum;
        //memanggil versi tes 1
        demo.tes(2.5);
        //Memanggil versi tes kedua
        jum=demo.tes(10);
        System.out.println("Hasil penjumlahan b + b =" +jum);
    }
}

```

3. Program menggunakan JUMLAH PARAMETER BERBEDA, TIPE SAMA

```

class OverloadMetode
{
    // Overload test for one integer parameter.
    void test(int a)
    {
        System.out.println("Nilai a: " + a);
    }

    // Overload test for two integer parameters.
    int test(int a, int b)
    {
        return a+b;
    }
}

```

// overload test for tree integer parameter

```

int test(int a,int b, int c)
{
    return a*b*c;
}
}

class Overload3
{
    public static void main(String args[])
    {
        OverloadMetode ob = new OverloadMetode();
        int jumlah,kali;
        // call all versions of test()
        ob.test(10);
        jumlah=ob.test(10,20) ;
        kali = ob.test(10,20,30);
        System.out.println("Hasil penjumlahan (10+20): " + jumlah);
        System.out.println("Hasil perkalian (10*20*30): " + kali);
    }
}

```

4. Berikut ini program menggunakan overloading konstruktor

```

class Man
{
    double gapok; double masakerja;
    // konstruktor untuk Man.
    Man(double g, double mk)
    {
        gapok=g; masakerja=mk;
    }
    // konstruktor yang tidak ditentukan ukurannya
    Man()
    {
        gapok = 0; masakerja = 0;
    }
}

```

}

```

// konstruktor digunakan ketika KaryawanKita dibuat
Man(double lembur)
{
    gapok=masakerja=lembur;
}

double hitung_gaji()
{
    return gapok*masakerja;
}
}

class OverloadingKonstruktor
{
public static void main(String args[])
{
    Man KaryawanKu = new Man(10,15);
    Man KaryawanMu = new Man();
    Man KaryawanKita = new Man(5);
    double gaji;
    gaji = KaryawanKu.hitung_gaji();
    System.out.println("Gaji Karyawanku adalah  = " + gaji);
    gaji = KaryawanMu.hitung_gaji();
    System.out.println("Gaji Karyawanmu adalah  = " + gaji);
    gaji = KaryawanKita.hitung_gaji();
    System.out.println("Gaji KaryawanKita adalah = " + gaji);
}
}

```

5. Berikut ini contoh program yang menerapkan konsep metode overriding menggunakan tipe pengembalian nilai.

[class superclass](#)

```

{

```

int p,l;

```

        superclass(int panjang,int lebar)
    {
        p=panjang;
        l=lebar;
    }

    int hitung()
    {
        return p*l;
    }
}

class subclass extends superclass
{
    int t;

    subclass(int panjang,int lebar,int tinggi)
    {
        super(panjang,lebar);
        t=tinggi;
    }

    {
        /*CARA 1
        int L;
        L=super.hitung();//MEMANGGIL METODE SUPERCLASS
        return L*t;
        */

        //CARA 2
        return p*l*t;
    }
}

```

```
class AplikasiMetodeOverriding
{
    public static void main(String[] args)
```

```

{
    subclass Objek=new subclass(3,4,6);
    System.out.println("LUAS "+Objek.hitung());
}

```

6. Program Simpan program Java di atas.
7. Eksekusi program Java Pertama dengan mengklik *Tool* □ *Compile Program Java*. Lalu
Tool □ *Menjalankan Program Java*.
8. Buat analisa terhadap proses program tersebut sebelum dijalankan dan pada saat dijalankan terhadap parameter dan argument yang digunakan..

V. Pertanyaan

1. Jelaskan apa yang dimaksud overloading & overriding
2. Apa perbedaan implementasi overloading pada metode dan konstruktor
3. Lakukan analisis dan buat kesimpulan terhadap penerapan overloading pada metode dan konstruktor.
4. Buat program penerapan overloading
5. Buat contoh program konversi otomatis pada java untuk semua variasi tipe. Berikan keterangan pada output untuk konversi yang tidak diijinkan.
6. Jelaskan perbedaan overriding dengan overloading

BAB 4

ABSTRACT & INTERFACE

I. Tujuan Instruksional Khusus

Setelah menyelesaikan praktek ini mahasiswa dapat :

1. menjelaskan pengertian *abstract & interface*,
2. mengimplementasikan kelas *abstract* dan *interface* kedalam program.

II. Persyaratan / Kebutuhan Sistem Komputer

Persyaratan / kebutuhan sistem untuk menjalankan program Java adalah sebagai berikut :

1. processor P-II atau yang lebih tinggi,
2. ram minimal 32 mbyte,
3. free space harddisk minimal 200 mbyte,
4. mouse (serial / ps2),
5. sistem operasi :
 - a. minimal Windows 98 (disarankan versi Windows yang lebih tinggi),
 - b. Linux.

III. Dasar Teori

Abstract

A simple definition of abstraction is that it displays only the relevant attributes of an object and hides any unnecessary details. As an example, when we are driving a car, we are only concerned with the basics, such as starting/ stopping the car, acceleration, and so on. An abstract example of this is this.

<https://thesassway.com/what-is-an-example-of-abstraction-in-computer-science/>

Metode yang tidak memerlukan implementasi sama sekali dinamakan abstrak. untuk mendeklarasikan digunakan kata kunci **abstrak**. Setiap kelas yang di dalamnya terdapat satu atau lebih metode abstrak harus dideklarasikan juga sebagai **kelas abstrak** dengan menambah kata kunci abstrak.

Kelas abstrak tidak dapat diinstansikan sehingga kita tidak diijinkan untuk membentuk objek dari kelas abstrak.

Namun kita masih diijinkan untuk mendeklarasikan **variable referensi**. Selanjutnya var referensi tsb dapat digunakan untuk menunjuk ke objek objek dari kelas turunan

Interface

Interface berbentuk seperti class dalam Java, tetapi tanpa variable dan metodenya dideklarasikan tanpa isi. Kita mendefinisikan *interface* tanpa menuliskan rincian implementasinya. Class-lah nanti yang akan merinci implementasi Interface. Sebuah class dapat mengimplementasikan beberapa interface sekaligus.

IV. Langkah Kerja

Jalankan Edit Plus, buat file baru java dan ketikkan kode program berikut :

1. Program untuk menampilkan serta menjumlahkan nilai i dan j

```
abstract class A
{
    int i;
    A(int i)
```

```
{
    this.i = i;
}
abstract public void tampil();
public int hitung()
{
    return i;
}
}
class B extends A
{
    int j;
    B(int i, int j)
    {
        super(i);
        this.j = j;
    }
    public void tampil()
    {
        System.out.println("Nilai j = " + j);
    }
}
class C extends A
{
    int j, k;
    C(int i, int j)
    {
        super(i);
        this.j = j;
    }
    public void tampil()
```

PAGE 2

{

k = i + j;

System.out.println("Nilai i + j = " + k);

```

    }
}
public class latihan
{
    public static void main(String[] args)
    {
        A lat;
A objek=new A();
        B x = new B(9, 2);
        C y = new C(9, 2);
        lat = x;
        System.out.println("Nilai i = " + lat.hitung());
        lat.tampil();
        lat = y;
        lat.tampil();
    }
}

```

9. Sebagai contoh di bawah ini diberikan sebuah program menggunakan interface.

```

interface Hitung
{
    double Luas();
}
class Lingkaran implements Hitung
{
    private double jari;
    public Lingkaran(double r)
    {
        this.jari = r;
    }
    public double Luas()
    {

```

double A;

A = jari*jari*22/7;

```
        return A;
    }
}

class SegiEmpat implements Hitung
{
    private double panjang;
    private double lebar;
    public SegiEmpat(double p, double l)
    {
        this.panjang = p;
        this.lebar = l;
    }
    public double Luas()
    {
        double L = panjang*lebar;
        return L;
    }
}

public class ClassAppl
{
    public static void main(String[] args)
    {
        SegiEmpat segiempat = new SegiEmpat(10,5);
        Lingkaran lingkaran = new Lingkaran(10);
        double A;
        A = segiempat.Luas();
        System.out.println("Luas segiempat = " + A);
        A = lingkaran.Luas();
        System.out.println("Luas lingkaran = " + A);
    }
}
```

10. Simpan program Java di atas.

11. Eksekusi program Java Pertama dengan mengklik **Tool** ▢ **Compile Program Java**. Lalu

Tool ▢ **Menjalankan Program Java**.

12. Buat analisa terhadap proses program tersebut sebelum dijalankan dan pada saat dijalankan terhadap parameter dan argument yang digunakan..

V. Pertanyaan

Buatlah *interface Animal* yang mempunyai dua metode: *eat* dan *move*. Semua metode ini tidak punya argumen atau nilai return. Method ini hanya mengeluarkan bagaimana object *Animal* makan dan bergerak. Sebagai contoh, seekor kelinci memakan wortel dan bergerak dengan melompat. Buat class *Fish* dan *Bear* yang menggunakan interface *Animal*. Terserah kepada Anda bagaimana menggunakan method *eat* dan *move*.

BAB 5

GRAPHIC USER INTEFACE (GUI)

I. Tujuan Instruksional Khusus

Setelah menyelesaikan praktek ini mahasiswa dapat :

1. menjelaskan apa yang dimaksud GUI,
2. menjelaskan apa yang dimaksud objek layer,
3. mampu menggunakan sejumlah class pendukung dalam pemrograman GUI.

II. Persyaratan / Kebutuhan Sistem Komputer

Persyaratan / kebutuhan sistem untuk menjalankan program Java adalah sebagai berikut :

1. processor P-II atau yang lebih tinggi,
2. ram minimal 32 Mbyte,
3. free space harddisk minimal 200 Mbyte,
4. mouse (Serial / PS2),
5. sistem operasi :
 - a. minimal Windows 98 (disarankan versi Windows yang lebih tinggi),
 - b. Linux.

III. Dasar Teori

GUI (*Graphical User Interface*) adalah suatu model antarmuka komputer yang menggunakan objek-objek grafis sebagai media interaksi dengan user. Model ini digunakan untuk menggantikan model klasik yang berbasis teks. Model ini biasanya lebih komunikatif, variatif, inovatif dan memiliki nilai jual tinggi. Ada beberapa hal yang menjadi pertimbangan ketika menentukan model antarmuka, yaitu:

1. Kecapatan proses
2. Klasifikasi pengguna
3. Kondisi perangkat komputer yang digunakan.

Objek layer adalah objek yang secara visual akan tampak dilayar sebuah window. Window ini bisa memiliki sebuah isi, seperti gambar, teks, button dan lain-lain. Java menganggap sebuah layar sebagai sebuah frame. Frame bisa berisi sejumlah komponen atau tidak berisi apa-apa. Frame dapat disetting sesuai dengan SO yang digunakan, dan dapat juga berbeda dari SO yang digunakan . Ini yang disebut sebagai konsep “*look and feel*”.

Java menyediakan dua kelas untuk membentuk sebuah window:

1. Frame
2. JFrame

Keduanya secara visual tidak ada bedanya, tetapi secara teknis JFrame memiliki kelebihan dibandingkan Frame, karena JFrame memiliki beberapa fitur untuk memudahkan kita memprogram window.

Frame

Frame terdapat pada paket *Abstract Window Toolkit*(AWT). Frame ini merupakan sebutan bagi objek window. Ada beberapa kondisi yang menjadi nilai default ketika sebuah objek frame dibuat:

1. Objek frame tidak akan terlihat dilayar sampai kita memberi perintah `setVisible(true)`. Penggunaan metode `show()` tidak diijinkan lagi.
2. Objek frame tidak memiliki title. Title disetting menggunakan metode `setTitle(String str)`.
3. Obek frame akan memunculkan tiga tombol control pada kanan atas, yaitu: Minimize, Maximize, dan Close.
4. Lokasi frame adalah (0,0) pada tepi kiri atas layer computer. Posisi ini bias diubah menggunakan `setLocation(int x,int y)`. Ukuran frame `setSize(int width,int height)`.
5. Layout manager yang digunakan adalah `BorderLayout`.

Frame turunan dari window class. Window dengan judul, menubar, border dan pengatur ukuran di pojok. Memiliki empat constructor , dua diantaranya memiliki penulisan seperti dibawah ini :

- `Frame()`
- `Frame(String title)`

`void setSize(int width, int height)` mengubah ukuran komponen ini dengan *width* dan *height* sebagai parameter.

`void setSize(Dimension d)` mengubah ukuran dengan *d.width* dan *d.height* berdasar pada spesifikasi *Dimension d*.

Default dari window adalah *not visible* atau tak tampak hingga Anda mengatur *visibility* menjadi *true*. Inilah *syntax* untuk method *setVisible*.

- `void setVisible(boolean b)`

Dalam mendesain aplikasi GUI, Object *Frame* selalu digunakan. Dibawah ini adalah contoh bagaimana membuat sebuah aplikasi.

```
import java.awt.*;

public class SampleFrame extends Frame {
    public static void main(String args[]) {
        SampleFrame sf = new SampleFrame();
        sf.setSize(100, 100); //Coba hilangkan baris ini
        sf.setVisible(true); //Coba hilangkan baris ini
    }
}
```

}

perhatikan bahwa tombol tutup pada frame tidak akan bekerja karena tidak ada mekanisme *event handling* yang ditambahkan di dalam aplikasi. Anda akan belajar tentang *event handling* pada modul selanjutnya.

JFrame

JFrame adalah komponen yang berfungsi sebagai Layar Utama (*Main Window*) untuk menjalankan aplikasi java GUI. Didalam JFrame dapat diletakkan komponen GUI sebagai interface, antara lain TextField, Button, Checkbox dan lainnya. Layar dapat ditampilkan dengan mengembangkan class JFrame sebagai berikut: setTitle(string teks) :memberi title pada window yang dibuat, setSize(int width,int height): memberi ukuran nyata dalam pixelpanjang kali lebar dari frame, setVisible(Boolean condition): menampilkan frame tersebut (nilai default adalah false atau tidakterlihat/ hidden).

Container

Container adalah objek yang berisi komponen GUI.Untuk menambahkan komponen pada frame maupun mengubah property Frame, program harus mengakses getContentPane() yang merupakan sebuah Container.

JLabel

JLabel adalah object yang biasa untuk menampilkan teks pada layar/ frame. Allignment adalah penempatan label pada bagian atas (*TOP*), bawah (*BOTTOM*), kiri (*LEFT*), kanan (*RIGHT*), tengah (*CENTER*).

TextField

TextField adalah komponen yang digunakan untuk memberikan masukan berupa text yang dapat diedit.

Password

Password adalah komponen yang mirip dengan TextField dengan perkecualian data yang dimasukan tidak ditampilkan di layar.

Button

JButton adalah GUI yang mempresentasikan sebuah tombol. Bila diaktifkan melalui ENTER atau diklik dengan Mouse maka JButton akan memberikan implus(trigger) berupa sebuah *event*.

IV. Langkah Kerja

1. Lakukan percobaan berikut sesuai teori diatas, lalu simpan dengan Filename Win1.

```
import javax.swing.*;

public class Win1 extends JFrame
{
    public Win1()
    {
        setTitle("Layar 1"); setSize(200,300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }
    public static void main(String[] args)
    {
        Win1 w = new Win1();
    }
}
```

2. Lakukan percobaan berikut sesuai teori diatas, lalu simpan dengan Filename Win2.

```
import javax.swing.*;

public class Win2
{
    JFrame w;
    public Win2()
    {
        w = new JFrame ();
        w.setTitle("layar 2");
        w.setSize(200,300);
        w.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        w.setVisible(true);
    }
    public static void main(String args[])
```

{

```

        new Win2();
    }
}

```

V. Latihan

1. Perhatikan Kode program diatas dan ubah kode program dengan menuliskan title mlangsung pada konstruktor. Simpan dengan Filename Win2b.
2. Ubah program JFrame dengan menambahkan Container dengan kode getContentPane() seperti kode dibawah ini lalu simpan dengan Filename Win3.

```

import java.awt.*;
import javax.swing.*;

public class Win3 extends javax.swing.JFrame
{
    public Win3()
    {
        JButton tombol = new JButton("Tombol");
        //Container c;
        getContentPane().add(tombol);
        setSize(300,80);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }

    public static void main(String[] args)
    {
        new Win3();
    }
}

```

3. Ubah program JFrame Win3 dengan menambahkan Label seperti kode dibawah ini lalu simpan dengan Filename MyLabel.

```

import javax.swing.*;
import java.awt.*;

```

```
public class mylabel  
{
```

```

JFrame layar;
JLabel L1, L2, L3, L4;
public mylabel()
{
    layar = new JFrame("Label Test");
    L2 = new JLabel ("Alamat   :");
    L1 = new JLabel ("Nama           :");
    Container c = layar.getContentPane();
    c.setLayout(new FlowLayout());
    L3 = new JLabel ("Kota           :",SwingConstants.LEFT);
    L4 = new JLabel ("Telepon   :",SwingConstants.RIGHT);
    c.add(L1);
    c.add(L2);
    c.add(L3);
    c.add(L4);
    layar.setSize(400,200);
    layar.setVisible(true);
}
public static void main (String argv[])
{
    new mylabel();
}
}

```

4. Ubah program MyLabel dengan menambahkan Label seperti kode dibawah ini lalu simpan dengan Filename/class name (MyTextField).

```

import javax.swing.*;
import java.awt.*;
public class MyTextField {
    private JFrame layar;
    private JTextField tf1;
    private JLabel L1,L2;
    private JPasswordField psw;

```

```
public MyTextField()
```

```

{
    layar = new JFrame("Text Field Test");
    L1 = new JLabel("User Name : ");
    L2 = new JLabel("Password : ");
    tf1 = new JTextField(25);
    JPasswordField psw = new JPasswordField(20);
    Container c = layar.getContentPane();
    c.setLayout( new FlowLayout());
    c.add(L1);
    c.add(tf1);
    c.add(L2);
    c.add(psw);
    layar.setSize(400,100);
    layar.setVisible(true);
}

public static void main(String [] args) {
    new MyTextField();
}
}

```

5. Ubah program MyTextField dengan menambahkan Label seperti kode dibawah ini lalu simpan dengan Filename/class name (MyButton).

```

//MyButton.java
import javax.swing.*;
import java.awt.*;
public class MyButton
{
    private JFrame layar; private
    JLabel L1, L2; private
    JTextField tf1; private
    JPasswordField psw;
    JButton b1, b2;
    public MyButton()

```

{

```

        layar = new JFrame("TextField Test");
        L1 = new JLabel("User Name: ");
        tf1 = new JTextField(25);
        L2 = new JLabel("Password: ");
        psw = new JPasswordField(25);
        b1 = new JButton("OK");
        b2 = new JButton("Cancel");
        Container c = layar.getContentPane();
        c.setLayout(new FlowLayout());
        c.add(L1);
        c.add(tf1);
        c.add(L2);
        c.add(psw);
        c.add(b1);
        c.add(b2);
        layar.setSize(900,80);
        layar.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        layar.setVisible(true);
    }

    public static void main(String[] args)
    {
        MyButton mb = new MyButton();
    }
}

```

BAB 6

ABSTRACT WINDOW TOOLKIT (AWT)

I. Tujuan Instruksional Khusus

Setelah menyelesaikan praktek ini mahasiswa dapat :

1. menjelaskan apa yang dimaksud AWT,
2. perbedaan antara komponen dan container,
3. mendesain aplikasi GUI menggunakan AWT.

4. Membuat tampilan yang komplek dalam mendesain aplikasi GUI.

II. Persyaratan / Kebutuhan Sistem Komputer

Persyaratan / kebutuhan sistem untuk menjalankan program Java adalah sebagai berikut :

1. processor P-II atau yang lebih tinggi,
2. ram minimal 32 Mbyte,
3. free space harddisk minimal 200 Mbyte,
4. mouse (Serial / PS2),
5. sistem operasi :
 - a. minimal Windows 98 (disarankan versi Windows yang lebih tinggi),
 - b. Linux.

III. Dasar Teori

The Java Foundation Class (JFC), merupakan bagian penting dari Java SDK, yang termasuk dalam koleksi dari API dimana dapat mempermudah pengembangan aplikasi JAVA GUI. JFC termasuk diantara 5 bagian utama dari API yaitu AWT dan Swing. Tiga bagian yang lainnya dari API adalah Java2D, *Accessibility*, dan Drag dan Drop. Semua itu membantu pengembang dalam mendesain dan mengimplementasikan aplikasi visual yang lebih baik.

AWT dan Swing menyediakan komponen GUI yang dapat digunakan dalam membuat aplikasi Java dan Applet. Anda akan mempelajari applet pada bab berikutnya. Tidak seperti beberapa komponen AWT yang menggunakan *native code*, keseluruhan Swing ditulis menggunakan bahasa pemrograman Java. Swing menyediakan implementasi *platform-independent* dimana aplikasi yang dikembangkan dengan platform yang berbeda dapat memiliki tampilan yang sama. Begitu juga dengan AWT menjamin tampilan *look and feel* pada aplikasi yang dijalankan pada dua mesin yang berbeda menjadi terlihat sama. Swing API dibangun dari beberapa API yang mengimplementasikan beberapa jenis bagian dari AWT. Jadi, komponen AWT dapat digunakan bersama komponen Swing.

Window classes fundamental

Dalam mengembangkan aplikasi GUI, komponen GUI seperti tombol atau textfield diletakkan di dalam kontainer. Berikut ini adalah daftar dari beberapa class penting pada kontainer yang telah disediakan oleh AWT.

Tabel 7.1 Daftar *class container AWT*

Class

AWT

Deskripsi

--	--

Komponen	Abstract Class untuk object yang console dan dapat ditampilkan pada berinteraksi dengan user. Bagian utama dari semua class AWT.
Kontainer	Abstract Subclass dari Class. Sebuah komponen yang dapat Component menampung
Panel	komponen yang lainnya. Turunan dari Container Class. Sebuah frame atau window tanpa titlebar, menubar tidak termasuk border. Superclass dari applet class.
Window	Turunan dari Container class. Top level window, dimana berarti tidak bisa dimasukkan dalam object yang lainnya. Tidak memiliki border dan menubar.
Frame	Turunan dari window class. Window dengan judul, menubar, border dan pengatur ukuran di pojok. Memiliki empat constructor, dua diantaranya memiliki penulisan seperti dibawah ini : Frame() Frame(String title)

Grafik

Beberapa metode grafik ditemukan dalam class *Graphic*. Dibawah ini adalah daftar dari beberapa metode.

-drawLine() -drawPolyline() -setColor()
 -fillRect() -drawPolygon() -getFont()
 -drawRect() -fillPolygon() -setFont()
 -clearRect() -getColor() -drawString()

Hubungan dari class ini adalah class *Color*, dimana memiliki tiga constructor.

<i>Format Constructor</i>	<i>Deskripsi</i>
Color(int r, int g, int b)	Nilai integer 0 - 255.
Color(float r, float g, float b)	Nilai float 0.0 - 1.0.
Color(int rgbValue)	Panjang nilai : 0 ke 224-1 (hitam ke putih).

Red: bits 16-23

Green: bits 8-15

Blue: bits 0-7

Komponen AWT

Berikut ini adalah daftar dari kontrol AWT. Kontrol adalah komponen seperti tombol atau textfield yang memungkinkan user untuk berinteraksi dengan aplikasi GUI. Berikut ini semua subclass dari class *Components*.

-Label -Button -Choice
 -TextField -Checkbox -List
 -TextArea -CheckboxGroup -Scrollbar

IV. Langkah Kerja

Jalankan Edit Plus, buat file baru java dan ketikkan kode program berikut :

1. Ketik contoh program dibawah ini menggunakan panel.

```
import java.awt.*;

public class GraphicPanel extends Panel {
    GraphicPanel() {
        setBackground(Color.black); //Konstanta dalam class Color
    }

    public void paint(Graphics g) { g.setColor(new
        Color(0,255,0)); //hijau g.setFont(new
        Font("Helvetica",Font.PLAIN,16));
        g.drawString("Hello GUI World!", 30, 100);
        g.setColor(new Color(1.0f,0,0)); //red
        g.fillRect(30, 100, 150, 10);
    }

    public static void main(String args[]) {
        Frame f = new Frame("Testing Graphics Panel");
        GraphicPanel gp = new GraphicPanel();
        f.add(gp);
        f.setSize(600, 300);
        f.setVisible(true);
    }
}
```

2. Ketik program aplikasi untuk membuat sebuah frame.

```
import java.awt.*;

class FrameWControls extends Frame { public
    static void main(String args[]) {
        FrameWControls fwc = new FrameWControls();
```

```
fwc.setLayout(new FlowLayout()); //akan dibahas lebih detail
```

pada pembahasan berikutnya

```
fwc.setSize(600, 600);
fwc.add(new Button("Test Me!"));
fwc.add(new Label("Labe"));
fwc.add(new TextField());
CheckboxGroup cbg = new CheckboxGroup();
fwc.add(new Checkbox("chk1", cbg, true));
fwc.add(new Checkbox("chk2", cbg, false));
fwc.add(new Checkbox("chk3", cbg, false));
List list = new List(3, false);
list.add("MTV");
list.add("V");
fwc.add(list);
Choice chooser = new Choice();
chooser.add("Dani");
chooser.add("Mulan Jamila");
chooser.add("Maya");
fwc.add(chooser);
fwc.add(new Scrollbar());
fwc.setVisible(true);
}
}
```

3. Simpan program Java di atas.
4. Eksekusi program Java Pertama dengan mengklik **Tool ▢ Compile Program Java**. Lalu

Tool ▢ Menjalankan Program Java.

5. Tampilkan hasilnya.

V. Pertanyaan

Tic-Tac-Toe

Buatlah tampilan GUI untuk program tic-tac-toe. Papannya terdiri dari enam kotak. Ingatlah bahwa Anda akan menambahkan kode ini pada tahap akhir untuk mengatasi interaksi antar

pengguna. Jadi, desainlah papan Anda dengan benar. Pastikanlah Anda memilih komponen

yang pantas untuk papan tersebut. keluarkan semua sisi artistik Anda. Anda dapat menggunakan AWT untuk latihan ini.

BAB 7

SWING

I. Tujuan Instruksional Khusus

Setelah menyelesaikan praktek ini mahasiswa dapat :

1. menjelaskan apa yang dimaksud Swing,
2. perbedaan antara komponen dan container,
3. mendesain aplikasi GUI menggunakan Swing,

4. membuat tampilan yang kompleks dalam mendesain aplikasi GUI.

II. Persyaratan / Kebutuhan Sistem Komputer

Persyaratan / kebutuhan sistem untuk menjalankan program Java adalah sebagai berikut :

1. processor P-II atau yang lebih tinggi,
2. ram minimal 32 Mbyte,
3. free space harddisk minimal 200 Mbyte,
4. mouse (Serial / PS2),
5. sistem operasi :
 - a. minimal Windows 98 (disarankan versi Windows yang lebih tinggi),
 - b. Linux.

III. Dasar Teori

Komponen *Swing*

Seperti pada package AWT, package dari Swing menyediakan banyak class untuk membuat aplikasi GUI. Package tersebut dapat ditemukan di *javax.swing*. Perbedaan utama antara keduanya adalah komponen Swing ditulis menyeluruh menggunakan Java. Program GUI ditulis menggunakan banyak class dari package *Swing yang mempunyai tampilan look and feel yang sama meski dijalankan pada platform yang berbeda*. Lebih dari itu, *Swing menyediakan komponen yang lebih menarik seperti color chooser dan option pane*. Nama dari komponen GUI milik Swing hampir sama persis dengan komponen GUI milik AWT. Perbedaan jelas terdapat pada penamaan komponen. Pada dasarnya, nama komponen Swing sama dengan nama komponen AWT tetapi dengan tambahan *huruf J pada prefixnya*. Sebagai contoh, satu komponen dalam AWT adalah button class. Sedangkan pada Swing, nama komponen tersebut menjadi JButton class. Berikut adalah daftar dari komponen Swing.

Tabel 6.1 Daftar komponen Swing

Komponen Swing	Penjelasan
JComponent	class induk untuk semua komponen Swing, tidak termasuk top-level kontainer
JButton	Tombol “push”. Berhubungan dengan class button dalam package AWT
JCheckBox	Item yang dapat dipilih atau tidak oleh pengguna. Berhubungan dengan class checkbox dalam package AWT

JFileChooser	Mengijinkan pengguna untuk memilih sebuah file. Berhubungan dengan class filechooser dalam package AWT
--------------	--

TextField	Mengijinkan untuk mengedit text satu baris. Berhubungan dengan class textfield dalam package AWT.
JFrame	Turunan dan Berhubungan dengan class frame dalam package AWT tetapi keduanya sedikit tidak cocok dalam kaitannya dengan menambahkan komponen pada kontainer. Perlu mendapatkan content pane yang terbaru sebelum menambah sebuah komponen.
JPanel	Turunan Jcomponent. Class Container sederhana tetapi bukan top-level. Berhubungan dengan class panel dalam package AWT.
JApplet	Turunan dan Berhubungan dengan class Applet dalam package AWT. Juga sedikit tidak cocok dengan class applet dalam kaitannya dengan menambahkan komponen pada container
JOptionPane	Turunan Jcomponent. Disediakan untuk mempermudah menampilkan pop-up kotak dialog.
JDialog	Turunan dan Berhubungan dengan class dialog dalam package AWT. Biasanya digunakan untuk menginformasikan sesuatu kepada pengguna atau prompt pengguna untuk input.
JColorChooser	Turunan Jcomponent. Memungkinkan pengguna untuk memilih warna yang diinginkan.

Setting Up Top-Level Containers

Seperti disebutkan diatas, top-level containers seperti *Jframe* dan *Japplet* dalam Swing sangat tidak cocok dengan AWT. Ini adalah syarat menambahkan komponen ke dalam kontainer. Jika Anda ingin menambahkan langsung sebuah komponen kedalam kontainer sebagai container AWT, pertama-tama Anda telah mendapatkan content pane dari kontainer. Untuk melakukan hal tersebut, Anda akan menggunakan method *getContentPane* dari container.

IV. Langkah Kerja

Jalankan Edit Plus, buat file baru java dan ketikkan kode program berikut :

1. Lakukan percobaan berikut sesuai teori diatas, lalu simpan dengan Filename Win1.

```
import javax.swing.*;

public class Win1 extends javax.swing.JFrame
{
    public Win1()
    {
        setTitle("Layar 1"); setSize(200,300);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

```

        setVisible(true);
    }
    public static void main(String[] args)
    {
        Win1 w = new Win1();
    }
}

```

2. Lakukan percobaan berikut sesuai teori diatas, lalu simpan dengan Filename Win2.

```

import javax.swing.*;
public class Win2
{
    JFrame w;
    public Win2()
    {
        w = new JFrame ();
        w.setTitle("layar 2");
        w.setSize(200,300);
        w.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        w.setVisible(true);
    }
    public static void main(String args[])
    {
        new Win2();
    }
}

```

3. Perhatikan Kode program diatas dan ubah kodeprogram dengan menuliskan title mlangsung pada konstruktor. Simpan dengan Filename Win2b.
4. Ubah program JFrame dengan menambahkan Container dengan kode getContentPane() seperti kode dibawah ini lalu simpan dengan Filename Win3.

```
import java.awt.*;  
import javax.swing.*;
```

```

public class Win3 extends javax.swing.JFrame
{
    public Win3()
    {
        JButton tombol = new JButton("Tombol");
        //Container c;
        getContentPane().add(tombol);
        setSize(300,80);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setVisible(true);
    }
    public static void main(String[] args)
    {
        new Win3();
    }
}

```

5. Ubah program JFrame Win3 dengan menambahkan Label seperti kode dibawah ini lalu simpan dengan Filename MyLabel.

```

import javax.swing.*;
import java.awt.*;

public class mylabel
{
    JFrame layar;

```

```
JLabel L1, L2, L3, L4;  
public mylabel()  
{
```

```

        layar = new JFrame("Label Test");
        L2 = new JLabel ("Alamat  :");
        L1 = new JLabel ("Nama      :");

        Container c = layar.getContentPane();
        c.setLayout(new FlowLayout());

        L3 = new JLabel ("Kota      :",SwingConstants.LEFT);
        L4 = new JLabel ("Telepon  :",SwingConstants.RIGHT);

        c.add(L1);
        c.add(L2);
        c.add(L3);
        c.add(L4);

        layar.setSize(400,200);
        layar.setVisible(true);
    }

    public static void main (String argv[])
    {
        new mylabel();
    }
}

```

6. Ubah program MyLabel dengan menambahkan Label seperti kode dibawah ini lalu simpan dengan Filename/class name (MyTextField).

```

import javax.swing.*;
import java.awt.*;

public class MyTextField {

```

PAGE 2

```
private JFrame layar;  
private JTextField tf1;  
private JLabel L1,L2;
```

```

private JPasswordField psw;

public MyTextField()
{
    layar = new JFrame("Text Field Test");
    L1 = new JLabel("User Name : ");
    L2 = new JLabel("Password : ");
    tf1 = new JTextField(25);
    JPasswordField psw = new JPasswordField(20);
    Container c = layar.getContentPane();
    c.setLayout( new FlowLayout());

    c.add(L1);
    c.add(tf1);
    c.add(L2);
    c.add(psw);

    layar.setSize(400,100);
    layar.setVisible(true);
}

public static void main(String [] args) {
    new MyTextField();
}
}

```

7. Ubah program MyTextField dengan menambahkan Label seperti kode dibawah ini lalu simpan dengan Filename/class name (MyButton).

//MyButton.java

PAGE 2

```
import javax.swing.*;  
import java.awt.*;  
public class MyButton
```

```

{
    private JFrame layar; private
    JLabel L1, L2; private
    JTextField tf1; private
    JPasswordField psw;
    JButton b1, b2;
    public MyButton()
    {
        layar = new JFrame("TextField Test");
        L1 = new JLabel("User Name: ");
        tf1 = new JTextField(25);
        L2 = new JLabel("Password: ");
        psw = new JPasswordField(25);
        b1 = new JButton("OK");
        b2 = new JButton("Cancel");
        Container c = layar.getContentPane();
        c.setLayout(new FlowLayout());
        c.add(L1);
        c.add(tf1);
        c.add(L2);
        c.add(psw);
        c.add(b1);
        c.add(b2);
        layar.setSize(900,80);
        layar.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        layar.setVisible(true);
    }

    public static void main(String[] args)
    {
        MyButton mb = new MyButton();
    }
}

```

1. Simpan program Java di atas.

2. Eksekusi program Java Pertama dengan mengklik Tool  Compile Program Java. Lalu Tool  Menjalankan Program Java.
3. Tampilkan hasilnya.

V. Pertanyaan

Buatlah tampilan GUI untuk program tic-tac-toe. Papannya terdiri dari enam kotak. Ingatlah bahwa Anda akan menambahkan kode ini pada tahap akhir untuk mengatasi interaksi antar pengguna. Jadi, desainlah papan Anda dengan benar. Pastikanlah Anda memilih komponen yang pantas untuk papan tersebut. keluarkan semua sisi artistik Anda. Anda dapat menggunakan Swing untuk latihan ini.

1. EXIT_ON_CLOSE (Menutup keseluruhan aplikasi)

```
setDefaultCloseOperation (JFrame. DO_NOTHING_ON_CLOSE);
```

2. DISPOSE (Menutup JFrame tanpa menutup aplikasi)

```
setDefaultCloseOperation (JFrame. DISPOSE_ON_CLOSE);
```

3. HIDE (Menyembunyikan JFrame)

```
setDefaultCloseOperation (JFrame. HIDE_ON_CLOSE);
```

4. DO_NOTHING (Tidak melakukan apapun)

```
setDefaultCloseOperation (JFrame. DO_NOTHING_ON_CLOSE);
```

Jika anda ingin mengganti default close operation-nya ada cukup menambahkan kode diatas di JFrame anda, letakkan dibawah initComponents();

BAB 8

MANAJEMEN LAYOUT

I. Tujuan Instruksional Khusus

Setelah menyelesaikan praktek ini mahasiswa dapat :

1. menjelaskan apa yang dimaksud objek container,
2. menjelaskan konsep layout manajemen,
3. menjelaskan tentang *flow layout*, *border layout*, dan *grid layout* dalam komponen GUI,
4. membuat tampilan yang kompleks dalam mendesain aplikasi GUI.

II. Persyaratan / Kebutuhan Sistem Komputer

Persyaratan / kebutuhan sistem untuk menjalankan program Java adalah sebagai berikut :

1. processor P-II atau yang lebih tinggi,
2. ram minimal 32 Mbyte,
3. free space harddisk minimal 200 Mbyte,
4. mouse (Serial / PS2),
5. sistem operasi :
 - a. minimal Windows 98 (disarankan versi Windows yang lebih tinggi),
 - b. Linux.

III. Dasar Teori

Objek kontainer adalah objek visual yang digunakan untuk menampung objek-objek visual lainnya termasuk objek container, untuk dikelompokkan menjadi satu grup. Objek container ini terdiri dari:

1. class Container,
2. class Panel,
3. class Jpanel.

Objek container dapat berisi objek container lainnya. Sedangkan pengaturan letak komponen didalam objek Container menggunakan layout manager.

Posisi dan ukuran suatu komponen ditentukan oleh layout manager. Layout manager mengatur tampilan dari komponen di dalam kontainer. Berikut ini beberapa layout manager yang terdapat di dalam Java.

- 1.FlowLayout
- 2.BorderLayout
- 3.GridLayout
- 4.GridBagLayout
- 5.CardLayout

Layout manager dapat diatur menggunakan method *setLayout* dari class *Container*. Method ini dapat ditulis sebagai berikut.

```
void setLayout(LayoutManager mgr)
```

Jika Anda memilih untuk tidak menggunakan layout manager, Anda dapat mengisi null sebagai

argumen untuk method ini. Tetapi selanjutnya, Anda akan mengatur posisi elemen secara manual dengan menggunakan method *setBounds* dari class Components.

```
public void setBounds(int x, int y, int width, int height)
```

Metode ini mengatur posisi berdasarkan pada argumen *x* dan *y*, dan ukuran berdasarkan argumen *width* dan *height*. Hal ini akan cukup menyulitkan dan membosankan untuk aplikasi jika Anda memiliki beberapa objek komponen didalam object container. Anda akan memanggil metode ini untuk setiap komponen.

FlowLayout manager

FlowLayout Manager adalah default manager untuk class *Panel* dan subclassnya, termasuk class *applet*. Cara meletakkan komponen dari FlowLayout Manager dimulai dari kiri ke kanan dan dari atas ke bawah, dimulai dari pojok kiri atas. Seperti pada saat Anda mengetik menggunakan editor kata pada umumnya. Berikut adalah bagaimana FlowLayout Manager bekerja, dimana memiliki tiga constructor seperti daftar di bawah ini.

FlowLayout constructors

FlowLayout() Membuat object baru FlowLayout dengan posisi di tengah dan lima unit horizontal dan vertical gap dimasukkan pada komponen sebagai default.

```
FlowLayout(int align)
```

Membuat objek baru FlowLayout dengan posisi spesifik dan lima unit horizontal dan vertikal gap dimasukkan pada komponen sebagai default.

FlowLayout constructors

FlowLayout(int align, int hgap, int vgap) Membuat object baru FlowLayout dengan argumen pertama sebagai posisi pada komponen dan *hgap* untuk horizontal dan *vgap* untuk vertikal pada komponen. Gap dapat dikatakan sebagai jarak antara komponen dan biasanya diukur dengan satuan pixel. Posisi argumen mengikuti penulisan sebagai berikut :

- 1.FlowLayout.LEFT
- 2.FlowLayout.CENTER
- 3.FlowLayout.RIGHT

BorderLayout manager

BorderLayout membagi kontainer menjadi lima bagian diantaranya utara, selatan, timur, barat, dan tengah. Setiap komponen dimasukkan ke dalam region yang spesifik. Region utara dan selatan membentuk jalur horizontal sedangkan region timur dan barat membentuk jalur vertikal.

Dan region tengah berada pada perpotongan jalur horizontal dan vertikal. Tampilan ini adalah

bersifat default untuk object *Window*, termasuk object dari subclass *Window* yaitu tipe *Frame* dan *Dialog*.

Constructor BorderLayout

BorderLayout()

Membuat object *BorderLayout* baru tanpa spasi yang diaplikasikan diantara komponen yang berbeda.

BorderLayout(int hgap, int vgap)

Membuat object *BorderLayout* baru dengan spasi unit *hgap* horizontal dan unit *vgap* vertikal yang diaplikasikan diantara komponen yang berbeda.

Seperti pada *FlowLayout Manager*, parameter *hgap* dan *vgap* disini juga menjelaskan jarak antara komponen dengan kontainer. Untuk menambahkan komponen kedalam region yang spesifik, gunakan method menambahkan dan melewati dua argumen yaitu : komponen yang ingin dimasukkan ke dalam region dan region mana yang ingin dipakai untuk meletakkan komponen. Perlu diperhatikan bahwa hanya satu komponen yang dapat dimasukkan dalam satu region. Menambahkan lebih dari satu komponen pada kontainer yang bersangkutan, maka komponen yang terakhir ditambahkan yang akan ditampilkan. Berikut ini adalah daftar dari kelima region.

1. *BorderLayout.NORTH*
2. *BorderLayout.SOUTH*
3. *BorderLayout.EAST*
4. *BorderLayout.WEST*
5. *BorderLayout.CENTER*

GridLayout manager

Dengan *GridLayout manager*, komponen juga diposisikan dari kiri ke kanan dan dari atas ke bawah seperti pada *FlowLayout manager*. *GridLayout manager* membagi kontainer menjadi baris dan kolom. Semua region memiliki ukuran yang sama. Hal tersebut tidak mempedulikan ukuran sebenarnya dari komponen.

Berikut ini adalah daftar dari constructor untuk class *GridLayout*.

Constructor GridLayout

GridLayout()

Membuat object GridLayout baru dengan satu baris dan satu kolom sebagai default

`GridLayout(int rows, int cols)`

Membuat object `GridLayout` baru dengan jumlah baris dan kolom sesuai dengan keinginan

`GridLayout(int rows, int cols, int hgap, int vgap)`

Membuat object `GridLayout` baru dengan jumlah baris dan kolom yang ditentukan. Unit spasi `hgap` horizontal dan `vgap` vertikal diaplikasikan ke dalam komponen.

Panel dan tampilan kompleks

Untuk membuat tampilan yang lebih lengkap, Anda dapat menggabungkan layout manager yang berbeda dengan menggunakan panel. Ingatlah bahwa panel adalah kontainer dan komponen pada saat yang sama. Anda dapat memasukkan komponen ke dalam panel dan kemudian menambahkan panel ke dalam region yang Anda inginkan di dalam kontainer.

IV. Langkah Kerja

Jalankan Edit Plus, buat file baru java dan ketikkan kode program berikut :

1. Bagaimanakah output dari program berikut?

```
import java.awt.*;

class FlowLayoutDemo extends Frame

{ public static void main(String args[]) {

    FlowLayoutDemo fld = new FlowLayoutDemo();
    fld.setLayout(new FlowLayout(FlowLayout.RIGHT, 10, 10));
    fld.add(new Button("ONE"));
    fld.add(new Button("TWO"));
    fld.add(new Button("THREE"));
    fld.setSize(100, 100);
    fld.setVisible(true);
}
}
```

2. Berikut ini adalah contoh program yang menunjukkan bagaimana *BorderLayout* bekerja.

~~PAGE~~

```
import java.awt.*;  
  
class BorderLayoutDemo extends Frame { public  
static void main(String args[]) { BorderLayoutDemo  
bld = new BorderLayoutDemo();
```

```

bld.setLayout(new BorderLayout(10, 10)); //may remove
bld.add(new Button("NORTH"), BorderLayout.NORTH);
bld.add(new Button("SOUTH"), BorderLayout.SOUTH);
bld.add(new Button("EAST"), BorderLayout.EAST);
bld.add(new Button("WEST"), BorderLayout.WEST);
bld.add(new Button("CENTER"), BorderLayout.CENTER);
bld.setSize(200, 200);
bld.setVisible(true);
}
}

```

3. Cobalah program ini.

```

import java.awt.*;

class GridLayoutDemo extends Frame { public
static void main(String args[]) {
    GridLayoutDemo gld = new GridLayoutDemo();
    gld.setLayout(new GridLayout(2, 3, 4, 4));
    gld.add(new Button("ONE"));
    gld.add(new Button("TWO"));
    gld.add(new Button("THREE"));
    gld.add(new Button("FOUR"));
    gld.add(new Button("FIVE"));
    gld.setSize(200, 200);
    gld.setVisible(true);
}
}

```

4. Perhatikan teknik yang digunakan pada contoh berikut.

```

import java.awt.*;

class ComplexLayout extends Frame {
public static void main(String args[]) {
    ComplexLayout cl = new ComplexLayout();
    Panel panelNorth = new Panel();

```

~~Page~~

Panel panelCenter = new Panel();

```

Panel panelSouth = new Panel();

/* Panel utara */

//Panel menggunakan FlowLayout sebagai default
panelNorth.add(new Button("ONE"));
panelNorth.add(new Button("TWO"));
panelNorth.add(new Button("THREE"));
/* Panel tengah */

panelCenter.setLayout(new GridLayout(4,4));
panelCenter.add(new TextField("1st"));
panelCenter.add(new TextField("2nd"));
panelCenter.add(new TextField("3rd"));
panelCenter.add(new TextField("4th"));
/* Panel selatan */

panelSouth.setLayout(new BorderLayout());
panelSouth.add(new Checkbox("Choose me!"),
BorderLayout.CENTER);
panelSouth.add(new Checkbox("I'm here!"),
BorderLayout.EAST);
panelSouth.add(new Checkbox("Pick me!"),
BorderLayout.WEST);

/* Menambahkan panel pada container Frame*/

//Frame menggunakan BorderLayout sebagai default
cl.add(panelNorth, BorderLayout.NORTH);
cl.add(panelCenter, BorderLayout.CENTER);
cl.add(panelSouth, BorderLayout.SOUTH);
cl.setSize(300,300);
cl.setVisible(true);
}
}

```

5. Simpan program Java di atas.
6. Eksekusi program Java Pertama dengan mengklik **Tool** ▢ **Compile Program Java**. Lalu

Tool ▢ **Menjalankan Program Java**.

7. Tampilkan hasilnya.

V. Pertanyaan

1. Jelaskan apa yang dimaksud objek container dan sebutkan macam-macam objek container.
2. Jelaskan apa fungsi layout manajemen, dan sebutkan macamnya.
3. Desain sebuah tampilan untuk input data mahasiswa

BAB 9

EVENT LISTENER

I. Tujuan Instruksional Khusus

Setelah menyelesaikan praktikum ini, diharapkan mahasiswa dapat:

1. mengetahui berbagai macam Action Listener dalam Java

2. memahami konsep event listener untuk merespon aktifitas user terhadap aplikasi yang dibuat.
3. mengimplementasikan action listener ke dalam program

II. Persyaratan / Kebutuhan Sistem Komputer

Persyaratan / kebutuhan sistem untuk menjalankan program Java adalah sebagai berikut :

1. processor P-II atau yang lebih tinggi,
2. ram minimal 32 Mbyte,
3. free space harddisk minimal 200 Mbyte,
4. mouse (Serial / PS2),
5. sistem operasi :
 - a. minimal Windows 98 (disarankan versi Windows yang lebih tinggi),
 - b. Linux.

III. Dasar Teori

Event handling

Event handling adalah suatu mekanisme yang menangani berbagai tindakan yang dilakukan oleh user terhadap **objek** yang ditampilkan window, agar program dapat merespon interaksi dari user tersebut. *Event handling* dalam java ditangani oleh **package java.awt.event.***. setiap event handling ditulis dalam class-class yang mengimplementasikan interface yang disediakan oleh java, atau disebut *event listener*.

Events adalah sebuah objek yang menggambarkan perubahan state dari sebuah source. Hal ini bisa disebabkan oleh interaksi user dengan elemen-elemen GUI. Beberapa aktifitas yang menyebabkan event dibangkitkan adalah penekanan tombol, memasukkan karakter melalui keyboard, memilih item dalam list, dan mengklik mouse. *Event* juga mungkin terjadi secara tidak langsung disebabkan oleh *timer expires*, *counter value*, terjadi kegagalan *software* atau *hardware*, operasi sudah lengkap.

Source adalah sebuah objek yang membangkitkan *event*. Ini terjadi ketika nilai *internal state* dari objek yang berubah. Source mungkin membangkitkan lebih dari satu tipe event. Sebuah source harus di register ke listener agar listener dapat menerima peringatan tentang tipe spesifik dari event. Setiap tipe dari event memiliki metode tersendiri.

Bentuk umum:

```
Public void addTypeListener(TypeListener e)
```

Type adalah nama dari event. e adalah sebuah reference ke event listener. Untuk contoh sebuah metode yang me-register sebuah tombol event listener:

```
Tbutton.addActionListener(eventclick)
```

Ketika event terjadi, semua register listener diperingatkan dan menerima kopian dari event

objek. Hal ini disebut juga sebagai multicasting event. dalam semua class, peringatan

dikirimkan hanya kepada listener yang terdaftar untuk menerima peringatan. Source juga menyediakan metode untuk menghilangkan register ke listener. Bentuk umum dari metode ini adalah:

```
Public void removeTypeListener(TypeListener e)
```

Type adalah nama dari event. e adalah refrence ke event listener. Contoh:

```
Tbutton.removeActionListener(eventclick)
```

Event listener

Event listener adalah sebuah konsep untuk merespon kegiatan yang terjadi didalam suatu objek.

Event listener merupakan implementasi penyampaian message antar objek. Objek yang mendapat message akan melakukan suatu action yang telah ditentukan didalam program.

Class yang menangkap suatu *event* lalu melakukan proses tertentu disebut *event-handler*. *Event handler* adalah class yang menerapkan *interface* dari suatu *event* tertentu.

Interface ActionListener

ActionListener merupakan event listener umum yang digunakan untuk merespon suatu event yang terjadi pada objek. Objek yang ingin merespon event ini harus terdaftar terlebih dahulu melalui metode `addActionListener(ActionListener)`.

Interface ChangeListener

Interface ini akan memantau kondisi objek, apakah mengalami perubahan state atau tidak. Perubahan state yang dimaksud contohnya adalah ketika objek di-click, ketika kursor mouse melewati objek tersebut, dan lainnya.

Metode yang diimplementasikan adalah `stateChanged(ChangeEvent e)`

Metode ini dieksekusi ketika sebuah objek mengalami perubahan.

Interface FocusListener

Interface ini akan memantau kondisi objek ketika objek tersebut menjadi focus atau kehilangan focus. Metode yang diimplementasikan adalah :

```
Void focusGained(FocusEvent e)
```

Dieksekusi ketika sebuah objek menjadi focus.

```
Void focusLost(FocusEvent e)
```

Dieksekusi ketika sebuah objek kehilangan focus.

Interface ItemListener

Interface ini akan memantau kondisi objek yang menampung sejumlah data ketika data-data tersebut diakses.

Metode yang digunakan adalah : void itemStateChanged(ItemEvent e).

Dieksekusi ketika sebuah data objek dalam objek diakses.

IV. Langkah Kerja

Jalankan Edit Plus, buat file baru java dan ketikkan kode program berikut :

1. Program untuk mengimplementasikan ActionListener

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class GUIEventHandlingActionListener extends JFrame implements ActionListener
{
    private JFrame frame = new JFrame();
    private JButton tbutton1 = new JButton("Submit");
    private JButton tbutton2 = new JButton("Exit");
    public GUIEventHandlingActionListener()
    {
        tbutton1.addActionListener(this);
        tbutton2.addActionListener(this);
        frame.getContentPane().setLayout(new FlowLayout());
        frame.getContentPane().add(tbutton1);
        frame.getContentPane().add(tbutton2);
        frame.pack(); frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public void actionPerformed(ActionEvent e)
    {
        Object obj = e.getSource();
        if (obj==tbutton1)
            JOptionPane.showMessageDialog(null, "Objek button 1");
```

else

```

        if (obj==tbutton2)
            System.exit(0);
            //JOptionPane.showMessageDialog(null, "Objek button 2");
    }

    public static void main(String[] args)
    {
        new GUIEventHandlingActionListener();
    }
}

```

2. Program untuk mengimplementasikan ChangeListener

```

import javax.swing.*;
import javax.swing.event.*;
import java.awt.*;
import java.awt.event.*;

public class GUIEventHandlingChangeListener extends JFrame implements
    ChangeListener
{
    private JFrame frame = new JFrame();
    private JButton tbutton = new JButton("Setuju");
    public GUIEventHandlingChangeListener()
    {
        tbutton.addChangeListener(this);
        frame.getContentPane().setLayout(new FlowLayout());
        frame.getContentPane().add(tbutton);
        frame.pack(); frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public void stateChanged(ChangeEvent e)
    {


---


        if (e.getSource()==tbutton)

```

```

        {
            JOptionPane.showMessageDialog(null, "Terjadi
            perubahan state pada
            buton 1");
        }

    }

    public static void main(String[] args)
    {
        new GUIEventHandlingChangeListener();
    }
}

```

3. Program untuk mengimplementasikan FocusListener

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class GUIEventHandlingFocusListener extends JFrame implements FocusListener
{
    private JFrame frame = new JFrame();
    private JLabel lKota = new JLabel("Kota :", JLabel.RIGHT);
    private JLabel lProp = new JLabel("Propinsi :", JLabel.RIGHT);
    private JTextField tKota = new JTextField();
    private JTextField tProp = new JTextField();
    private JButton tbutton1 = new JButton("Submit");
    private JButton tbutton2 = new JButton("Exit");
    public GUIEventHandlingFocusListener()
    {
        tKota.addFocusListener(this); tProp.setEnabled(false);
        frame.getContentPane().setLayout(new GridLayout(2,3));
        frame.getContentPane().add(lKota);
        frame.getContentPane().add(tKota);
    }
}

```

```
frame.getContentPane().add(tbutton1);  
frame.getContentPane().add(lProp);
```

```

        frame.getContentPane().add(tProp);
        frame.getContentPane().add(tbutton2);
        frame.pack();
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void focusGained(FocusEvent e)
    {
    }

    public void focusLost(FocusEvent e)
    {
        String str = tKota.getText().toUpperCase();
        //if (!str.isEmpty())
        //{
            if (str.equals("JAKARTA"))
                tProp.setText("DKI");
            else
                if (str.equals("SEMARANG"))
                    tProp.setText("JATENG");
                else
                    if (str.equals("BANDUNG"))
                        tProp.setText("JABAR");

            //}
        }

    public static void main(String[] args)
    {
        new GUIEventHandlingFocusListener();
    }
}

```

4. Program untuk mengimplementasikan ItemListener

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.awt.event.*;
```

```
public class GUIEventHandlingItemListener extends JFrame implements ItemListener
```

```

{
    private String[] listKota = {"SURABAYA","SEMARANG","BANDUNG"};
    private String[] listProp = {"JATIM","JATENG","JABAR"};
    private JFrame frame = new JFrame();
    private JLabel lKota = new JLabel("Kota :", JLabel.RIGHT);
    private JLabel lProp = new JLabel("Propinsi :", JLabel.RIGHT);
    private JComboBox cbKota = new JComboBox(listKota);
    private JTextField tProp = new JTextField();
    public GUIEventHandlingItemListener()
    {
        cbKota.addItemListener(this); tProp.setEnabled(false);
        frame.getContentPane().setLayout(new GridLayout(2,2));
        frame.getContentPane().add(lKota);
        frame.getContentPane().add(cbKota);
        frame.getContentPane().add(lProp);
        frame.getContentPane().add(tProp);
        frame.pack(); frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    public void itemStateChanged(ItemEvent e)
    {
        int index = cbKota.getSelectedIndex();
        String str = listProp[index];
        tProp.setText(str);
    }
    public static void main(String[] args)
    {
        new GUIEventHandlingItemListener();
    }
}

```

5. Program untuk mengimplementasikan MenuListener

```

import javax.swing.*;
import javax.swing.event.*;
import java.awt.*;
import java.awt.event.*;

public class GUIEventHandlingMenuListener extends JFrame implements MenuListener
{
    private JFrame frame = new JFrame();
    private JMenuBar menu = new JMenuBar();
    private JMenu file = new JMenu("File");
    private JMenu edit = new JMenu("Edit");
    private JMenu help = new JMenu("Help");
    private JMenuItem mi1 = new JMenuItem("New");
    private JMenuItem mi2 = new JMenuItem("Save");
    private JMenuItem mi3 = new JMenuItem("Exit");
    public GUIEventHandlingMenuListener()
    {
        file.addMenuListener(this);
        menu.add(file);
        menu.add(edit);
        menu.add(help);
        frame.getContentPane().add(menu);
        frame.pack(); frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }

    public void menuCanceled(MenuEvent e)
    {
    }

    public void menuDeselected(MenuEvent e)
    {
    }

    public void menuSelected(MenuEvent e)
    {
        if (e.getSource() == file)

```

```

        JOptionPane.showMessageDialog(null, "Menu File di-select");
        file.add(mi1);
        file.add(mi2);
        file.add(mi3);
    }

    public static void main(String[] args)
    {
        new GUIEventHandlingMenuListener();
    }
}

```

6. Simpan program Java di atas.
7. Eksekusi program Java Pertama dengan mengklik *Tool* □ *Compile Program Java*. Lalu
Tool □ *Menjalankan Program Java*.
8. Tampilkan hasilnya.

V. Pertanyaan

1. Jelaskan konsep kerja dari event listener.
2. Sebutkan beberapa interface dari event listener dan jelaskan.
3. Lengkapi program input data mahasiswa pada modul 13 dengan event listener.

BAB 10

PEMROGRAMAN DATABASE

<https://blog.rumahcoding.co.id/membuat-aplikasi-crud-menggunakan-java-swing-dan-mysql-part-1/>

<https://blog.rumahcoding.co.id/membuat-aplikasi-crud-menggunakan-java-swing-dan-mysql-part-3/>

I. Tujuan Instruksional Khusus

Setelah menyelesaikan praktikum ini, diharapkan mahasiswa dapat:

1. membuat program Java interaktif yang bisa membaca input dari keyboard

2. menggunakan BufferedReader untuk mendapatkan input dari keyboard melalui layer console
3. menggunakan JOptionPane untuk mendapatkan input dari keyboard melalui GUI
4. membuat program pengolahan data dengan menggunakan teks dan database

II. Persyaratan / Kebutuhan Sistem Komputer

Persyaratan / kebutuhan sistem untuk menjalankan program Java adalah sebagai berikut :

1. processor P-II atau yang lebih tinggi,
2. ram minimal 32 Mbyte,
3. free space harddisk minimal 200 Mbyte,
4. mouse (Serial / PS2),
5. sistem operasi :
 - a. minimal Windows 98 (disarankan versi Windows yang lebih tinggi),
 - b. Linux.

III. Dasar Teori

Pengolahan data merupakan suatu jenis pemrograman yang sangat penting dalam mempelajari bahasa pemrograman. Untuk melakukan pengolahan data menggunakan java selain GUI diperlukan juga paket java.io agar dapat memanfaatkan input / output stream.

Menggunakan BufferedReader untuk mendapatkan input

Pada bagian ini, kita akan menggunakan kelas BufferedReader yang berada di java.io package untuk mendapatkan input dari keyboard.

Berikut ini adalah langkah-langkah yang diperlukan untuk mendapatkan input dari keyboard :

- a. Tambahkan dibagian bagian paling atas code :
 Import java.io.*;
- b. Tambahkan statement berikut :
 BufferedReader dataIn = new BufferedReader(new InputStreamReader(System.in));
- c. Deklarasi variable String temporer untuk mendapatkan input dan gunakan fungsi readLine() untuk mendapatkan input dari keyboard. Anda harus mengetikkannya didalam blok try-catch :

```

Try
{
String temp = dataIn.readLine();
}
Catch(IOException e)
```

~~PAGE~~

{

System.out.println("Error in getting input")

```
}
```

Menggunakan JOptionPane untuk mendapatkan input

Cara lain untuk mendapatkan input dari user adalah dengan menggunakan kelas JOptionPane yang didapatkan dari javax.swing package. JOptionPane membuat kemudahan dengan memunculkan dialog box standar yang memberikan kepada user sebuah nilai untuk menginformasikan sesuatu.

IV. Langkah Kerja

1. Jalankan Edit Plus, buat file baru java dan ketikkan kode program berikut :

Menampilkan kata dari user menggunakan BufferedReader

```
import java.io.*;
class GetInputBufferedReader
{
    public static void main(String[] args)
    {
        BufferedReader dataIn=new BufferedReader(new
        InputStreamReader(System.in));
        String word[] = new String[3];
        try
        {
            System.out.print("Enter word1 : ");
            word[0]=dataIn.readLine();
            System.out.print("Enter word2 : ");
            word[1]=dataIn.readLine();
            System.out.print("Enter word3 : ");
            word[2]=dataIn.readLine();
        }
        catch (IOException e)
        {
            System.out.println("Error!");
        }
    }
}
```

```
}
```

```
System.out.println(word[0]+" "+word[1]+" "+word[2]);
```

```

    }
}

```

2. Menampilkan kata dari user menggunakan JOptionPane

```

import javax.swing.*;
class GetInputJOption
{
    public static void main(String[] args)
    {
        String word[]= new String[3];
        word[0]=JOptionPane.showInputDialog("Enter word1");
        word[1]=JOptionPane.showInputDialog("Enter word2");
        word[2]=JOptionPane.showInputDialog("Enter word3");
        String msg=word[0]+" "+word[1]+" "+word[2];
        JOptionPane.showMessageDialog(null,msg);
    }
}

```

3. Memasukkan input data dari user

```

import java.io.*;
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
class GUIInput
{
    public static void main(String[] args)
    {
        final JFrame frame=new JFrame("INPUT DATA");
        JLabel INim=new JLabel("NIM");
        JLabel INm=new JLabel("NAMA");
        JLabel lAlm=new JLabel("ALAMAT");
        JLabel lJur=new JLabel("JURUSAN");
        JLabel lProdi=new JLabel("PRODI");
        final JTextField tNim=new JTextField(15);

```

```
final JTextField tNm=new JTextField(15);
```

```

final JTextField tAlm=new JTextField(15);
final JTextField tJur=new JTextField(15);
final JTextField tProdi=new JTextField(15);
JButton tSimpan=new JButton("Simpan");
JButton tExit=new JButton("Exit");
tSimpan.addActionListener(new ActionListener()
{
    public void actionPerformed(ActionEvent e)
    {
        if (tNim.getText().trim().equals(""))
        {
            JOptionPane.showMessageDialog(frame,"NIM    Masih    Kosng","Simpan
data",JOptionPane.WARNING_MESSAGE);
            tNim.requestFocus();
        }
        else
        {
            FileInputStream
            fin;
            BufferedReader
            din;
            FileOutputStream
            fout; try
            {
                fin=new FileInputStream("mhs.txt");
                din=new        BufferedReader(new
InputStreamReader(fin));

                String isidata[]=new
                String[5]; String
                teks,thisLine,mergeLine;
                mergeLine="";
                int i=0;
                while ((thisLine=din.readLine())!=null)

```

```
{
```

```
isidata=thisLine.split(",");
```

```
if
```

```
(isidata[0].equals(tNim.getText().trim()))
```

```

        {
            i=1;

        }
        else
        {
            mergeLine += thisLine +

            "\n";
        }
    }

    fin.close();
    if (i==1)
    {
        JOptionPane.showMessageDialog(frame,"NIM sudah ada","Cari
data",JOptionPane.WARNING_MESSAGE);
    }
    else
    {
        FileOutputStream("mhs.txt");
        PrintStream(fout).print(mergeLin
e);

        new

        new

        PrintStream(fout).println(tNim.getText()+" "+tNm.getText()+" "+tAlm.getText()+" "+
+tJur.getText()+" "+tProdi.getText());

        fout.close();
        tNim.setText("");
    }
    tNm.setText("");
    tAlm.setText("");
    tJur.setText("");
    tProdi.setText("");
    tNim.requestFocus();

```

}

catch (Exception exc)

```

        {
            System.err.println("Salah:
"+exc);
        }
    }
}
});

```

```

Container konten; konten=frame.getContentPane();
konten.setLayout(new GridBagLayout());
GridBagConstraints pos=new GridBagConstraints();
pos.anchor=GridBagConstraints.WEST;
pos.gridx=5;
pos.gridy=5;
konten.add(lNim,pos);
pos.gridx++;
konten.add(tNim,pos);
pos.gridy++;
pos.gridx=5;
konten.add(lNm,pos);
pos.gridx++;
konten.add(tNm,pos);
pos.gridy++;
pos.gridx=5;
konten.add(lAlm,pos);
pos.gridx++;
konten.add(tAlm,pos);
pos.gridy++;
pos.gridx=5;
konten.add(lJur,pos);
pos.gridx++;
konten.add(tJur,pos);
pos.gridy++;
pos.gridx=5;

```

```

        konten.add(lProdi,pos);
        pos.gridx++;
        konten.add(tProdi,pos);
        pos.gridy++;
        pos.gridx=5;
        konten.add(tSimpan,pos);
        pos.gridx++;
        konten.add(tExit,pos);
        frame.pack();
        frame.setVisible(true);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

4. Menggunakan Database MySql

```

import java.awt.*;
import java.awt.event.*;
import java.io.*;
import javax.swing.*;
import java.sql.*;

public class GuiInputDatabaseMysql
{
    public static void main(String[] args)
    {
        JFrame f=new JFrame();
        JLabel lNomor=new JLabel("NOMOR");
        JLabel lJudul=new JLabel("JUDUL");

        JTextField tNomor=new JTextField(15);
        JTextField tJudul=new JTextField(15);
    }
}

```

try

J
B
u
t
t
o
n
t
S
i
m
p
a
n
=
n
e
w
J
B
u
t
t
o
n
(
"
S
a
v
e
"
)
;
J
B
u
t
t
o
n
t
E
x
i
t
=
n
e
w
J
B

u
t
t
o
n
(
"
E
x
i
t
"
)
;

```

    {
        //cara 1 menggunakan ODBC
// Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");//step 1
        //cara 2
        Class.forName("com.mysql.jdbc.Driver");//step 1
    }
    catch (ClassNotFoundException cnfe)
    {
        System.err.println(cnfe);
    }

    tSimpan.addActionListener(new ActionListener()
    {
        public void actionPerformed(ActionEvent e)
        {
            String sql="INSERT INTO buku VALUES("
            +tNomor.getText()+","
            +tJudul.getText()+")";

            if (tNomor.getText().trim().equals(""))
            {
                JOptionPane.showMessageDialog
                (f,"nomor masih kosong","simpan
data",JOptionPane.WARNING_MESSAGE);
                tNomor.requestFocus();
            }
            else
            {
                try

                {
                    //cara 1 //Connection
                    conn=DriverManager.getConnection("jdbc:odbc:KonekDsn","root","password");
                    //KoneksDsn=nama data source name
                    //cara 2 Connection

                    conn=DriverManager.getConnection("jdbc:mysql://localhost/perpus","root","password"
);
                    // con
                    =DriverManager.getConnection("jdbc:mysql://localhost/akademik","root","password");

```

Statement stmt=conn.createStatement();//step 4

```

        stmt.executeUpdate(sql);

        tNomor.setText("");
        tJudul.setText("");
        tNomor.requestFocus();
    }

    catch (Exception exc)
    {
        System.err.println("salah:"+exc);
    }

    }

});

Container konten;
konten=f.getContentPane();
konten.setLayout(new GridBagLayout());
GridBagConstraints pos=new GridBagConstraints();
pos.anchor=GridBagConstraints.WEST;
pos.gridx=5;
pos.gridy=5;

konten.add(lNomor,pos);
pos.gridx++;
konten.add(tNomor,pos);
pos.gridy++;
pos.gridx=5;

konten.add(lJudul,pos);
pos.gridx++;
konten.add(tJudul,pos);
pos.gridy++;
pos.gridx=5;

konten.add(tSimpan,pos);
pos.gridx++;
konten.add(tExit,pos);

f.pack(); f.setVisible(true);
f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

}
}

```

5. Simpan program Java di atas.
6. Eksekusi program Java Pertama dengan mengklik *Tool* □ *Compile Program Java*.

Lalu *Tool* □ *Menjalankan Program Java*.

7. Tampilkan hasilnya.

V. Pertanyaan

Buat program input data dari user ke dalam database untuk data mahasiswa, dosen dan matakuliah.

DAFTAR PUSTAKA

<file:///F:/JavaReference/javanotes4.1/c1/index.html>

Glenn L. Vanderburg. et al, 1996, Tricks of the Java Programming Gurus, First Edition, Sams.net Publishing, Indianapolis

Jamie Jaworski, 1998, Java 1.2 Unleashed, Macmillan Computer Publishing, Indianapolis

Lorin, H., 1975, Sorting and Sort System, Addison Wesley Co., Menlo Park, California.

Michael Morrison, et al.,1997, Java Unleashed, Second Edition,Sams.net Publishing,
Indianapolis

Schildt,H.,2002,Java™ 2:The Complete Reference,Fifth Edition,McGraw-Hill/Osborne

Sedgewick,R.,1983,Algoritm, Addison Wesley Co.,Menlo Park,California.